



QueueVault Initial Audit Report

Version 0.1

Cyfrin.io

April 27, 2026

QueueVault Audit Report

Gabal

April 27, 2026

QueueVault Audit Report

Prepared by: Gabal

- Gabal

Table of contents

See table

- QueueVault Audit Report
- Table of contents
- About Gabal
- Risk Classification
- Audit Details
 - Scope
- Protocol Summary
 - Roles
- Executive Summary
 - Issues found
- Findings
 - High
 - * [H-1] Incorrect Accounting Leads to Insolvency
- Medium Findings

About Gabal

I am security researcher specializing in smart contract auditing and Web3 security. focus includes identifying high-impact vulnerabilities such as accounting inconsistencies, reentrancy vectors, and economic exploits in DeFi protocols.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

Audit Details

Scope

```
1 src/  
2 --- QueueVault.sol
```

Protocol Summary

QueueVault is a vault-based protocol that allows users to deposit ERC20 tokens and receive shares representing their ownership. Withdrawals are processed through a request queue requiring manager approval.

Roles

- **User:** Deposits assets and receives shares
- **Manager:** Approves withdrawal requests

Executive Summary

Issues found

Severity	Number of issues found
High	2
Medium	3
Low	0
Info	0
Gas Optimizations	0
Total	5

Findings

High

[H-1] Incorrect Accounting Leads to Insolvency

Description:

The protocol uses an internal variable (`totalDeposited`) instead of the actual token balance. This causes desynchronization between recorded and real assets, especially with fee-on-transfer tokens.

Impact:

- Vault insolvency
- Failed withdrawals
- Incorrect accounting

Proof of Concept:

```
1 function testPoc1_IncorrectAccounting_FeeOnTransferBreaksSolveny()
  external {
2   FeeOnTransferToken token = new FeeOnTransferToken();
3   QueueVault vault = new QueueVault(address(token), manager);
4
5   token.mint(alice, 100 ether);
```

```
6     vm.startPrank(alice);
7     token.approve(address(vault), type(uint256).max);
8     vault.deposit(100 ether);
9     vm.stopPrank();
10
11     assertEq(token.balanceOf(address(vault)), 90 ether);
12     assertEq(vault.totalAssets(), 100 ether);
13
14     uint256 aliceShares = vault.balanceOf(alice);
15     vm.prank(alice);
16     uint256 id = vault.requestWithdraw(aliceShares);
17     vm.prank(manager);
18     vault.approveRequest(id);
19
20     vm.expectRevert();
21     vm.prank(alice);
22     vault.claim(id);
23 }
```

Recommended Mitigation: Use actual token balance instead of internal accounting:

```
return asset.balanceOf(address(this));
```

[H-2] Share Mispricing Allows Inflation

Description: Shares are calculated using user input instead of the actual received tokens, allowing over-minting when interacting with non-standard ERC20 tokens.

Impact:

Share inflation Value dilution Economic exploits

Proof of Concept:

```
1
2 function testPoc2_ShareMispricing_UsingInputAmountNotReceivedAmount()
   external {
3     FeeOnTransferToken token = new FeeOnTransferToken();
4     QueueVault vault = new QueueVault(address(token), manager);
5
6     token.mint(alice, 100 ether);
7     vm.startPrank(alice);
8     token.approve(address(vault), type(uint256).max);
9     uint256 shares = vault.deposit(100 ether);
10    vm.stopPrank();
11
12    assertEq(shares, 100 ether);
13    assertEq(token.balanceOf(address(vault)), 90 ether);
14 }
```

Recommended Mitigation: Calculate shares using actual received tokens:

uint256 received = balanceAfter - balanceBefore;

Medium Findings

[M-1] Unaccounted Token Donations

Description: Direct transfers to the vault bypass accounting, creating inconsistencies.

Impact:

Hidden funds Incorrect share pricing

Proof of Concept:

```
1
2 function testPoc3_DirectDonation_UnaccountedAssets() external {
3     MockERC20 token = new MockERC20("Mock", "MOCK");
4     QueueVault vault = new QueueVault(address(token), manager);
5
6     token.mint(alice, 100 ether);
7     token.mint(bob, 10 ether);
8
9     vm.startPrank(alice);
10    token.approve(address(vault), type(uint256).max);
11    vault.deposit(100 ether);
12    vm.stopPrank();
13
14    vm.prank(bob);
15    token.transfer(address(vault), 10 ether);
16
17    assertEq(token.balanceOf(address(vault)), 110 ether);
18    assertEq(vault.totalAssets(), 100 ether);
19 }
```

Recommended Mitigation: Rely on balanceOf() instead of internal variables or introduce a sync mechanism.

[M-2] Reentrancy in Deposit Function

Description: External token calls occur before state updates, allowing reentrancy.

Impact:

Nested deposits Unexpected execution flows

Proof of Concept:

```
1
2 function testPoc4_ReentrancySurface_DepositCanReenter() external {
3     ReentrantToken token = new ReentrantToken();
```

```
4     QueueVault vault = new QueueVault(address(token), manager);
5
6     token.mint(address(token), 20 ether);
7     vm.prank(address(token));
8     token.approve(address(vault), type(uint256).max);
9
10    token.setAttack(vault, 10 ether);
11
12    vm.prank(address(token));
13    vault.deposit(10 ether);
14
15    assertEq(vault.balanceOf(address(token)), 20 ether);
16 }
```

Recommended Mitigation: Use ReentrancyGuard and follow checks-effects-interactions pattern.

[M-3] Manager Zero Address Misconfiguration

Description: Constructor does not validate manager address.

Impact:

Withdrawals permanently blocked

Proof of Concept:

```
1
2 function testPoc5_ManagerZeroAddress_BlocksApprovals() external {
3     MockERC20 token = new MockERC20("Mock", "MOCK");
4     QueueVault vault = new QueueVault(address(token), address(0));
5
6     token.mint(alice, 50 ether);
7     vm.startPrank(alice);
8     token.approve(address(vault), type(uint256).max);
9     vault.deposit(50 ether);
10    uint256 id = vault.requestWithdraw(vault.balanceOf(alice));
11    vm.stopPrank();
12
13    vm.expectRevert(bytes("not manager"));
14    vm.prank(manager);
15    vault.approveRequest(id);
16 }
```

Recommended Mitigation: Validate constructor inputs:

`require(_manager != address(0)); require(_asset != address(0));`